

Python CLI Developer Challenge

Build A Product: Split-Expense CLI

Project Overview

Split-Expense CLI is a command-line tool for tracking shared expenses among a small group (roommates, a trip, a household). One person pays for something and logs who owes a share. The people who owe can see what they owe and flag that they've paid. The person who paid confirms or disputes each claim.

Required Features

1. Users can create an account and log in.
2. A user who pays for something (the "payer") can add a shared expense, specifying the amount, a description, and who owes a share.
3. Any user who's part of a shared expense can see a list of all shared expenses they're involved in.
4. A user can view the details of a shared expense (their computed share, description, date) and submit a settlement claim, that is, flag that they've paid their share.
5. The payer can view the settlement claims submitted against an expense they created.
6. The payer can accept or reject a settlement claim.

Optional Features

1. Users only see and settle expenses shared with people on their contacts list.
 2. Users are notified of new decisions (a claim accepted or rejected) the next time they run a command. This is a deliberate downgrade from the original's "real time notifications": a CLI has no open connection to push through, so "real time" honestly means "checked on next launch," not a live socket. Pretending otherwise would be the app lying about what it does.
-

Preparation Guidelines

Steps to take before building:

1. Create a **GitHub repository**, add a README, and clone it to your computer.
2. Add contributors

Challenge 1: Command Surface & Help Text

Expected Length to Complete: 1 week

Due Date: set per cohort calendar

Challenge Summary

Build the full command-line interface with `argparse`: every command and subcommand should exist and have accurate `--help` output. The logic underneath is stubbed, hardcoded or fake return values only. No data structures, no persistence, no real auth yet.

NB

- Create a pull request to get review and feedback on the command surface before moving on to real logic.
- Use only the Python standard library's `argparse` for this challenge, no Click, no Typer. The point is to feel what a full framework abstracts away before you reach for one.
- Do not copy an existing CLI's command structure wholesale.

Guidelines

1. On trello, create issues to set up the command surface:
 - a. `register` and `login` commands (stubbed).
 - b. Commands to: list all shared expenses, view a single expense's details, add an expense, view settlement claims on an expense you created, accept or reject a claim.
 - c. A `profile` command that, at minimum, shows (stub data for now): how many expenses the user has paid for, how many they've participated in, and a list of both.
2. Create issues for any other necessary tasks, tag each as a feature, bug, or chore.
3. On a feature branch, build all commands into a `cli/` package with accurate `--help` text for every command and subcommand.

Challenge 2: In-Memory Core Logic, Linting & Tests

Expected Length to Complete: 1 day

Due Date: set per cohort calendar

Challenge Summary

Implement real logic behind the Challenge 1 command stubs, backed by in-memory Python data structures (no file, no database yet).

NB

- Create a pull request to get review and feedback when this challenge is done.

Tools

- Linting Library: Pylint
- Style Guide: PEP 8
- Testing Framework: PyTest

Guidelines

1. On Trello, create issues to implement and test, using in-memory data structures:
 - List all shared expenses
 - Get a single expense's details
 - Create a shared expense
 - Submit a settlement claim
2. Create issues for any other necessary tasks (feature, bug, or chore).
3. Implement the logic behind each Challenge 1 stub using Python lists/dicts or dataclasses.
4. Set up Pylint and follow PEP 8.
5. Set up PyTest.
6. Version your CLI with semantic versioning, starting at `v0.1.0`. This replaces URL versioning, same idea (don't silently break your users), CLI-native mechanism: bump it in `pyproject.toml`, expose it via `expensecli --version`.
7. Using separate branches per feature, implement each command for real.
8. At minimum, the following should work:

Command	Functionality
<code>expensecli list</code>	List all shared expenses in memory
<code>expensecli show</code> <code><expense_id></code>	Show a single shared expense's details
<code>expensecli add</code>	Create a shared expense

```
expensecli claim  
<expense_id>
```

Submit a settlement claim on your
share

9. Write tests for the logic behind each command, not the argument-parsing plumbing.
10. Integrate coverage reporting (`coverage.py` + Codecov, Coveralls works too) with a badge in the README.